Backend Engineer - Product Importer

1. Objective

- Acme Inc. needs a functional web application that can import products from a CSV file (approximately 500,000 records) into a SQL database. The app should be designed for scalability and optimized performance when handling large datasets.

2. Specification

a. STORY 1 — File Upload via UI

- As a user, I should be able to upload a large CSV file (up to 500,000 products) directly through the application's user interface.
    1. The UI should display a clear and intuitive file upload component.
    2. During the upload process, the UI should show a **real-time progress indicator** (e.g., percentage, progress bar).
    3. If duplicate products exist, the system should **automatically overwrite** based on SKU, treating the SKU as case-insensitive.
    4. The SKU must remain unique across all records.
    5. Products may be marked as active or inactive (even though this field is not part of the CSV).
    6. The upload flow should be optimized for handling large files efficiently while remaining responsive.

b. STORY 1A — Upload Progress Visibility

- As a user, I should be able to **see the upload progress** directly in the UI in real time.
    1. The progress should dynamically update as the file is being processed.
    2. The UI should display visual cues like a progress bar, percentage, or status messages (e.g., "Parsing CSV", "Validating", "Import Complete").

3. If the upload fails or encounters errors, the UI should clearly show the failure reason and provide a retry option.
4. The technical implementation may rely on APIs (e.g., SSE, WebSockets, or polling), but the focus is on providing a **smooth, interactive visual experience**.

c. **STORY 2 — Product Management UI**
   - As a user, I should be able to **view, create, update, and delete products** entirely from a web interface.
     1. The interface should support:
     2. Filtering by SKU, name, active status, or description.
     3. Paginated viewing of product lists with clear navigation controls.
     4. Inline editing or a simple modal form for creating/updating products.
     5. Deletion with a confirmation step.
     6. Minimalist, clean design — even a simple HTML/JS frontend is sufficient as long as it demonstrates all functional capabilities.

d. **STORY 3 — Bulk Delete from UI**
   - As a user, I should be able to **delete all existing products** directly from the UI.
     1. This operation must be protected with a confirmation dialog (e.g., “Are you sure? This cannot be undone.”).
     2. The UI should display success/failure notifications after the operation.
     3. This feature should be responsive and provide visual feedback during processing.

e. **STORY 4 — Webhook Configuration via UI**
   - As a user, I should be able to **configure and manage multiple webhooks** through the application’s user interface.
     1. The UI should allow adding, editing, testing, and deleting webhooks.
     2. It should display webhook URLs, event types, and enable/disable status.

3. There should be visual confirmation of successful test triggers (e.g., response code, response time).
4. The webhook processing should remain performant and not degrade overall application responsiveness.

3. Toolkit

a. Acme Inc. is also opinionated about their tech stack. The tools should be:
   - Web framework: Python based frameworks (Flask, Tornado, Django, FastAPI)
   - Asynchronous execution: Celery/Dramatiq with RabbitMQ/Redis
   - ORM: SQLAlchemy (if not django).
   - Database: We recommend PostgreSQL (and works with the deployment choice below).
   - Deployment: The application should be hosted on a publicly accessible platform of your choice (e.g., Heroku, Render, AWS, GCP, etc.). You can use any free-tier or easily deployable option that allows reviewers to access and test the app.

4. Next Steps

a. Build the app
b. Push the code to Github/Bitbucket/Gitlab
c. Deploy the app
   - **Note:** Feel free to use any AI tools of your choice and share the output of all prompts in any file format or link.
   - **Submission Guidelines:** Please reply to the same email thread within 24 hours.

5. Points and Rating scheme

a. APPROACH AND CODE QUALITY
   - Code quality is a very important part of the assignment. Better documented, standards compliant code that is readable wins over brilliant hacks. Remember CPU and Memory are cheap and expendable, humans are not.

b. COMMIT HISTORY
    - Clean commits and a good commit history offers a sneak peak into planning and execution.
c. DEPLOYMENT
    - Gone are the days when deployment was handled by a separate team. Modern engineers are expected to manage infrastructure as code and take ownership of deploying their applications. Hosting your app on any public platform (such as Heroku, Render, AWS, or GCP) is a great way to showcase how your solution performs in a real-world environment.
d. TIMEOUT FOR LONG OPERATIONS
    - For example, platforms like Heroku have a 30-second timeout limit. The product upload process is expected to exceed this limit, so your solution should handle long-running operations using asynchronous workers or other suitable approaches. Your implementation should address this elegantly. Feel free to ask if you'd like ideas or suggestions.